Nama : Khafid Zaifuddin Rohman
Kelas : TI-1H
No. Presensi : 16

2.1. PERCOBAAN 1 — PERULANGAN FOR

1. Komponen perulangan for

   Contoh:

```
for (int i = 0; i < 10; i++) {
   // perintah yang diulang
}
```

   - Inisialisasi: int i = 0 membuat penghitung dan menetapkan nilai awalnya.
   - Kondisi: i < 10 membuat perulangan berjalan selama kondisi bernilai benar.
   - Update: i++ menambah penghitung satu kali setiap selesai menjalankan isi perulangan.
   - Isi perulangan: perintah untuk membaca dan memeriksa nilai mahasiswa.

2. Inisialisasi tertinggi dengan 0 dan terendah dengan 100

   Nilai mahasiswa berada pada rentang 0–100. Variabel tertinggi dimulai dari nilai terendah yang mungkin agar nilai masukan dapat menggantikannya bila lebih tinggi. Sebaliknya, variabel terendah dimulai dari nilai tertinggi yang mungkin agar nilai masukan dapat menggantikannya bila lebih rendah.

   Jika tertinggi dimulai dari 100 dan terendah dari 0, nilai masukan yang valid tidak akan lebih dari 100 atau kurang dari 0. Akibatnya, keduanya tetap pada nilai awal dan hasilnya salah.

3. Fungsi dan alur kode pencarian nilai tertinggi dan terendah

```
if (nilai > tertinggi) {
   tertinggi = nilai;
}
if (nilai < terendah) {
   terendah = nilai;
}
```

   Setiap kali nilai mahasiswa dibaca, program membandingkannya dengan nilai tertinggi dan terendah saat ini. Jika lebih besar dari tertinggi, variabel tersebut diperbarui. Jika lebih kecil dari terendah, variabel tersebut juga diperbarui. Setelah semua nilai diproses, kedua variabel menyimpan nilai tertinggi dan terendah dari masukan.

4. Menghitung mahasiswa lulus dan tidak lulus

   Deklarasikan penghitung sebelum perulangan:

```
int jumlahLulus = 0;
int jumlahTidakLulus = 0;
```

Setelah nilai dibaca di dalam perulangan, periksa batas kelulusan:

```
if (nilai >= 60) {
   jumlahLulus++;
} else {
   jumlahTidakLulus++;
}
```

Setelah perulangan, tampilkan hasilnya:

```
System.out.println("Nilai tertinggi: " + tertinggi);
System.out.println("Nilai terendah: " + terendah);
System.out.println("Jumlah mahasiswa lulus: " + jumlahLulus);
System.out.println("Jumlah mahasiswa tidak lulus: " + jumlahTidakLulus);
```

2.2. PERCOBAAN 2 — PERULANGAN WHILE

1. Maksud dan kegunaan sintaks
   a. nilai < 0 || nilai > 100

      Operator || berarti “atau”. Kondisi bernilai benar jika nilai kurang dari 0 atau lebih dari 100. Dengan demikian, program dapat mengenali nilai yang tidak berada dalam rentang 0–100.

   b. continue

      continue melewati sisa perintah pada iterasi saat ini dan melanjutkan ke pemeriksaan kondisi perulangan berikutnya. Pada pemeriksaan nilai, ini membuat program meminta input ulang untuk mahasiswa yang sama. Karena itu, i++ harus dilewati ketika nilai tidak valid.

2. Alasan i++ ditulis di akhir perulangan

   i++ di akhir perulangan membuat penghitung bertambah setelah satu nilai valid selesai diproses. Jika input tidak valid dan program menjalankan continue, kenaikan penghitung dilewati sehingga input berikutnya tetap untuk mahasiswa yang sama.

   Jika i++ dipindahkan ke awal isi perulangan, penghitung bertambah sebelum nilai diperiksa. Input yang tidak valid dapat ikut dihitung sebagai satu mahasiswa, sehingga program mungkin berhenti sebelum menerima jumlah nilai valid yang diminta.

3. Jumlah perulangan jika jumlah mahasiswa 19

   Jika semua input valid, perulangan berjalan 19 kali, satu kali untuk setiap mahasiswa. Jika terdapat input tidak valid, program meminta input ulang, sehingga jumlah percobaan input bisa lebih dari 19.

4. Menambahkan pesan untuk nilai A

   Letakkan pesan tambahan di dalam cabang kategori A:

```
if (nilai > 80) {
   System.out.println("Kategori nilai: A");
   System.out.println("Bagus, pertahankan nilainya");
} else if (nilai > 73) {
   System.out.println("Kategori nilai: B+");
}
```

   Karena validasi dilakukan terlebih dahulu, kategori A mencakup nilai di atas 80 sampai 100.

2.3. PERCOBAAN 3 — PERULANGAN DO-WHILE

1. Jika nama pertama yang dimasukkan adalah "batal", berapa kali perulangan dilakukan?

   Perulangan dilakukan satu kali. Isi do dijalankan terlebih dahulu, baru kondisi diperiksa. Ketika nama "batal" terdeteksi, break menghentikan perulangan.

2. Kondisi berhenti perulangan

   Perulangan dihentikan ketika nama pelanggan sama dengan "batal", lalu program menjalankan break. Jika perbandingan menggunakan equalsIgnoreCase, huruf besar dan kecil tidak memengaruhi hasilnya.

3. Fungsi nilai true pada kondisi do-while

   while (true) membuat kondisi perulangan selalu benar, sehingga perulangan terus berjalan. Program keluar melalui break ketika kondisi berhenti, yaitu nama pelanggan "batal", terpenuhi.

4. Mengapa perulangan tetap berjalan tanpa inisialisasi dan update?

   do-while tidak memerlukan variabel penghitung. Pada contoh ini, setiap pengulangan meminta nama pelanggan baru. Nama tersebut menjadi masukan untuk menentukan apakah transaksi dilanjutkan atau program dihentikan. Perulangan berjalan karena kondisinya true, dan berhenti melalui break saat menerima "batal".